

# Vorlesung 1: Softwarekonstruktion

Prof. Dr. Falk Howar

Automated Quality Assurance  
Lehrstuhl für Software Engineering  
Fakultät für Informatik  
TU Dortmund



## Softwarekonstruktion (Ingenieurmäßiges Vorgehen):

- Aufgaben, Methoden, Werkzeuge und Anspruch des Ingenieurs

## Softwarekonstruktion.

- Die Herausforderungen des Themenfeldes Softwarekonstruktion **aufzählen** und **erläutern**



- Die Aspekte der Softwarekonstruktion **benennen**



- Die Geschichte des Software Engineering **wiedergeben**



- Die Prinzipien ingenieurmäßigen Vorgehens **wiedergeben** und **erläutern**



# Agenda

## 1. Software Konstruktion

- Kurze Geschichte des Software Engineering, Teil I
- Ingenieurwissenschaft
- Kurze Geschichte des Software Engineering, Teil II

## 2. Bestandsaufnahme Software Konstruktion

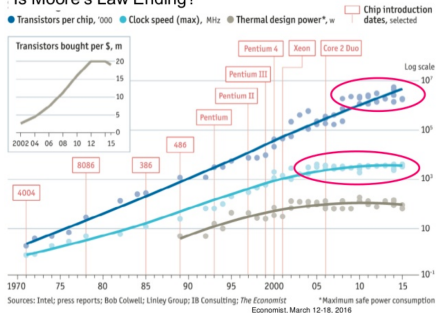
## 3. Besonderer Fokus: Software Qualität

## 4. Begleitendes Beispiel

# Moore's Law und Software

- Erste programmierbare Computer Ende 1940er Jahren
- Erste Programmiersprachen in den 1950er Jahren
- Ab den 1960ern wird Hardware immer günstiger und leistungsfähiger
- Software Projekte werden größer

## Is Moore's Law Ending?



# Software Krise (I)

## Ende der 1960er Jahre

- Software Projekte laufen unbefriedigend ab
- Projekte sprengen Budgets
- Deadlines werden nicht eingehalten
- Software hat Fehler und ist langsam
- Projekte werden abgebrochen

# Software Krise (II)

*The major cause of the software crisis is that the machines have become several orders of magnitude more powerful! To put it quite bluntly: as long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem, and now we have gigantic computers, programming has become an equally gigantic problem.*

*Edsger Dijkstra, The Humble Programmer (EWD340),  
Communications of the ACM*

# Software Krise (III)

Die Idee einer Disziplin „Software Engineering“ entsteht bei einer NATO Konferenz 1968 / 1969 in einer Diskussion der Software Krise

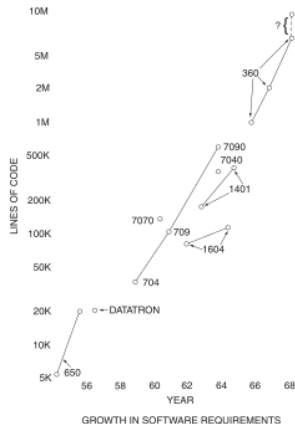
»Production of large software has become a scare item for management. By reputation it is often an unprofitable morass, costly and unending.

...

No less a person than T. J. Watson said that OS/360 cost IBM over 50 million dollars a year during its preparation, and at least 5000 man-years' investment.

...

The commitment to many software projects has been withdrawn. This is indeed a frightening picture.◀



<http://homepages.cs.ncl.ac.uk/brian.randell/NATO/nato1968.PDF>



# Software Engineering

Software Engineering is...

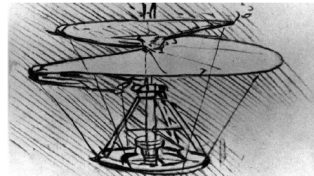


... the establishment and use of **sound engineering principles** in order to obtain **economically** software that is **reliable and works efficiently** on real machines. [NR68]

... the **systematic approach** to the **development, operation, maintenance, and retirement** of software." [IEEE83]



- **Fachleute auf dem Gebiet der Technik**
- Technik: von Menschen gemachte Gegenstände und deren Herstellung und Verwendung
- Das Wort Ingenieur kommt vom lateinischen "ingenium" Bedeutung ungefähr: **Erfindung, Scharfsinn**
- Der Begriff ist im 16ten Jahrhundert entstanden und hat ursprünglich den **Beruf des künstlerischen Erfinders** beschrieben (z.B. Leonardo da Vinci)



Quelle teilw.: [http://www.iaeng.org/about\\_IAENG.html](http://www.iaeng.org/about_IAENG.html)

# Ingenieure



„Engineers *apply the knowledge of the mathematical and natural sciences* (biological and physical), *with judgment and creativity to develop ways to utilize the materials and forces of nature* for the benefit of mankind. The subjects are diverse and include names

*like bioengineering, computer engineering, electrical and electronics engineering, financial engineering, industrial engineering, internet engineering and systems engineering, etc.*“

# Ingenieure: Aufgaben und Anspruch

## Ingenieure konstruieren komplexe technische Systeme

- Oft interdisziplinär (Physik, Chemie, Biologie)
- Auch für neue Produkte: traditionell vorhandene Methoden und Mittel zur Herstellung
- Ingenieur beherrscht Herstellungsmethoden, Werkzeuge und Werkstoffe (Regeln, Standards, Erfahrungswissen)

## Dreifacher Anspruch

- **Produkt:** „utilize the materials and forces of nature“
  - Technischer Anspruch
- **Methoden:** „with judgment and creativity“
  - Empirischer Anspruch
- **Ingenieur:** „for the benefit of mankind“
  - Ethischer Anspruch

# Praxis: Abwägung Kosten / Nutzen

## Das Teufelsquadrat im Projektmanagement

- Optimierung einer Dimension geht zu Lasten der anderen Dimensionen
- Generell unmöglich, alle Dimensionen zu optimieren

## Strategien für die Abwägung von Kosten und Nutzen

- Schätzen
- Messen



Maße  
Metriken



Teufelsquadrat nach Harry Sneed

# Praxis: Beherrschung von Komplexität

## Strategien für die Beherrschung von Komplexität

### **Abstraktion:**

- Einsatz von geeigneten Modellen
- Ausblenden von Detailinformation

### **Werkzeuge (Automatisierung):**

- Erhöhung von Leistungsfähigkeit und Präzision

### **Spezialisierung / Modularisierung:**

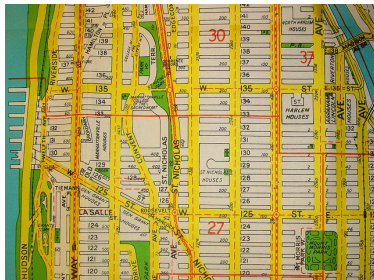
- Aufteilung in klar abgegrenzte Unterstrukturen
- Partitionierung der Aufgaben (Dekomposition)

### **Standardisierung:**

- Dokumentation von Erfahrungswissen
- Einfache Integration und Wiederverwendung

# Abstraktion

**Abstrakt (Plan / Modell):**



**Konkret:**



<https://www.flickr.com/photos/23465812@N00/7968397600/>  
<https://www.flickr.com/photos/eirikref/2302281782/>  
<https://creativecommons.org/licenses/by/2.0/>

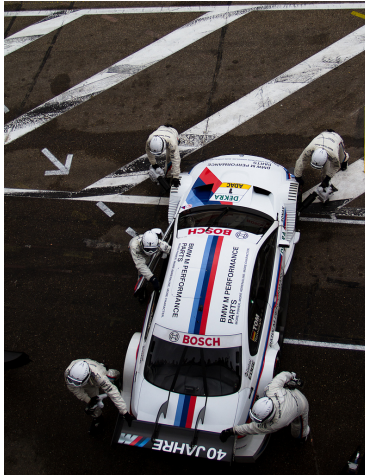
# Werkzeuge (Automatisierung)



<https://flic.kr/p/fkmB7T>, (CC BY 2.0)

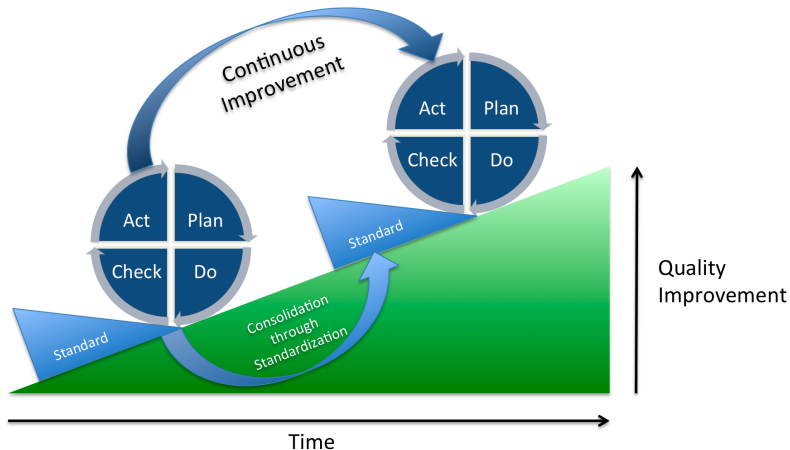


# Spezialisierung / Modularisierung



<https://www.flickr.com/photos/martindew/7945608932/>  
<https://creativecommons.org/licenses/by/2.0/>

# Standardisierung



**Beispiele:** Stecker, Schrauben, Gewinde, Vorgehen, Maße, ...

# Zusammenfassung Ingenieure

- Konstruieren komplexe technische Systeme
- Nutzen Wissen aus den Naturwissenschaften
- Dreifacher Anspruch  
(Produkt, Methoden, Ingenieur bzw. Technisch, Empirisch, Ethisch)
- Wägen Kosten gegen Nutzen ab (Schätzen und Messen)
- Benötigen Strategien zum Umgang mit Komplexität (Abstraktion, Spezialisierung / Modularisierung, Standardisierung, Werkzeuge)

# Software Engineering (II)

Abstraktion,  
Spezialisierung,  
Modularisierung,  
Standardisierung,  
...



## Software Engineering is...

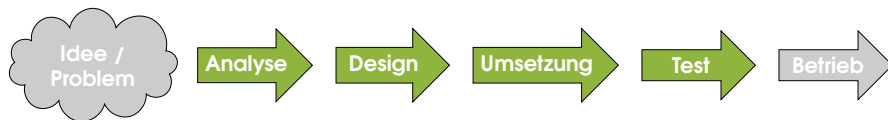
... the establishment and use of **sound engineering principles** in order to obtain economically software that is reliable and works efficiently on real machines. [NR68]

Schätzen, Messen, ...



... the systematic approach to the development, operation, maintenance, and retirement of software." [IEEE83]

# Software Konstruktion



*Methoden, Sprachen, Vorgehen*

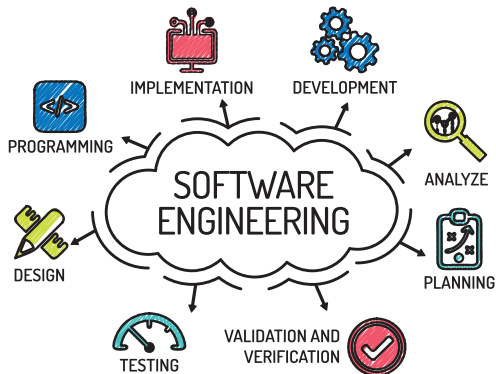
*Sprachen, Modelle*

*Werkzeuge*

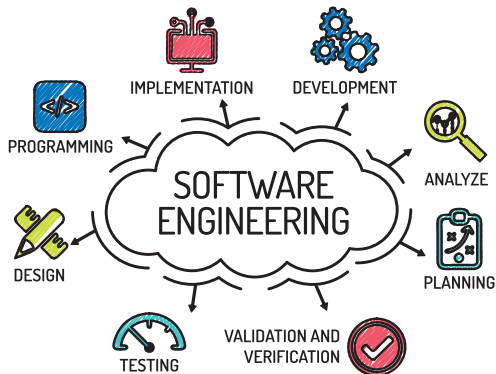
*Standards*

*Qualität, Kennzahlen (Produkt / Prozess)*

# SWK Teilgebiete (Spezialisierung)



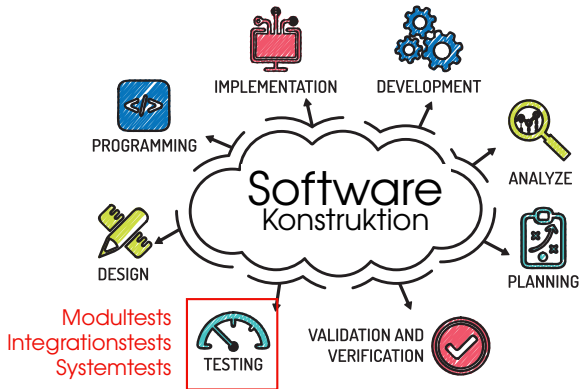
# SWK Teilgebiete (Spezialisierung)



## Software Engineering:

Marketing, Organisation, Multi-Projekt Management, SWK, Wartung, Support, Dokumentation, ...

# SWK Teilgebiete (Spezialisierung)



## Software Engineering:

Marketing, Organisation, Multi-Projekt Management, SWK, Wartung, Support, Dokumentation, ...



# SWK Geschichte (Werkzeuge)

## Auszug:

- 1968 Dijkstra "goto" ist gefährlich
- frühe 1970er Strukturierte Programmierung, erste Objektorientierung
- späte 1970er Erste IDEs
- späte 1980er CASE Werkzeuge
- späte 1980er Weitere Verbreitung von OOP durch C++ und OOD
- frühe 1990er Kommerzielle Werkzeuge für Anforderungsmanagement
- später 1990er Java, UML, ...
- frühe 2000er Eclipse IDE, CASE Werkzeuge verschwinden

# Beispiel: Sprachen (Komplexität)

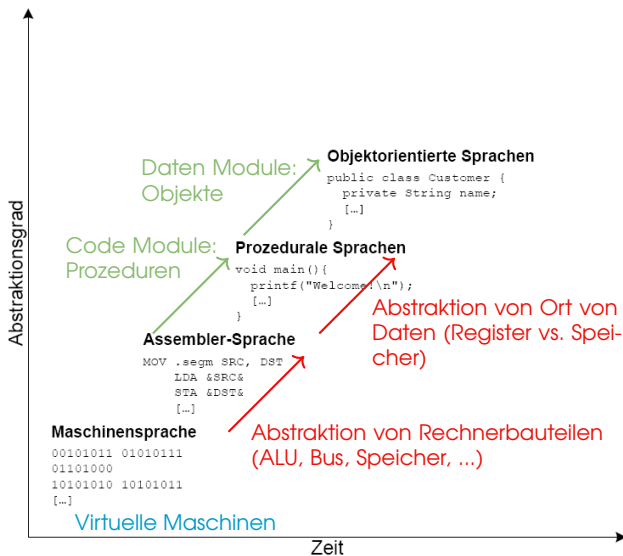
- Problem:  
Komplexitäts-  
Beherrschung

- Lösung:

**Abstraktion**

**Modularisierung**

**Standards**



# Zusammenfassung Geschichte

- Neue Technologien (z.B. Web)
- Neue Klassen von Anwendungen (z.B. Apps)

→ Neue Paradigmen, Werkzeuge und Methoden (z.B. SCRUM)

→ Domänen-spezifische Lösungen (Kampfjet vs. Web-Anwendung)

## **ABER:**

Generell gültige Ansprüche an Produkt, Methoden und Ingenieur

# Anspruch an Software

- Korrektheit
- Zuverlässigkeit
- Sicherheit
- Effizienz
- Benutzbarkeit
- Wartbarkeit
- Lesbarkeit
- Erweiterbarkeit
- Wiederverwendbarkeit

# Anspruch an Methoden

- Stellt Anforderungen an Software sicher
- Effektiv
- Effizient
- Planbar



# Anspruch an Software Ingenieur

## Software Engineering Code of Ethics and Professional Practice

ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices

### PREAMBLE

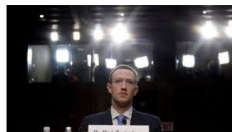
The short version of the code summarizes aspirations at a high level of the abstraction; the clauses that are included in the full version give examples and details of how these aspirations change the way we act as software engineering professionals. Without the aspirations, the details can become legalistic and tedious; without the details, the aspirations can become high sounding but empty; together, the aspirations and the details form a cohesive code.

Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

1. PUBLIC – Software engineers shall act consistently with the public interest.
2. CLIENT AND EMPLOYER – Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. PRODUCT – Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
4. JUDGMENT – Software engineers shall maintain integrity and independence in their professional judgment.
5. MANAGEMENT – Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
6. PROFESSION – Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
7. COLLEAGUES – Software engineers shall be fair to and supportive of their colleagues.
8. SELF – Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

# Facebook / Cambridge Analytica

- Cambridge Analytica „klaut“ Nutzerdaten von Facebook
- Erstellt psychologische und politische Profile der Nutzer (Nix: 4.000 bis 5.000 Datenpunkte zu jedem Erwachsenen US Amerikaner)
- Profile werden Basis für Werbekampagnen vor US Wahl 2016
- Trump wird überraschend Präsident



Fakt oder Fiktion?

# Zusammenfassung Software Konstruktion

- Software Konstruktion zielt ab auf die **ingenieurmäßige** Entwicklung, Wartung, Anpassung und Weiterentwicklung **großer Softwaresysteme**.
- Verwendung **bewährter systematischer Vorgehensweisen**, Prinzipien, Methoden und Werkzeuge.
- Empirisches Vorgehen (**Ausprobieren, Beobachten, Standardisieren**)
- Methoden und Vorgehen teilweise spezifisch für Domäne.
- Berücksichtigung der Aspekte: **Funktionalität, Kosten, Zeit, Qualität**.



# Agenda

## 1. Software Konstruktion

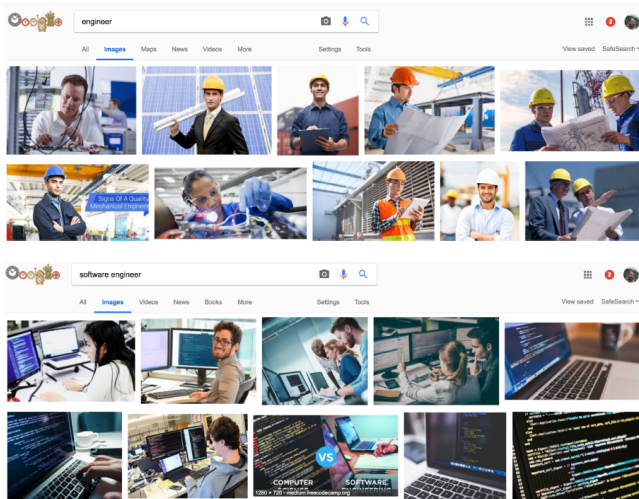
- Kurze Geschichte des Software Engineering, Teil I
- Ingenieurwissenschaft
- Kurze Geschichte des Software Engineering, Teil II

## 2. Bestandsaufnahme Software Konstruktion

## 3. Besonderer Fokus: Software Qualität

## 4. Begleitendes Beispiel

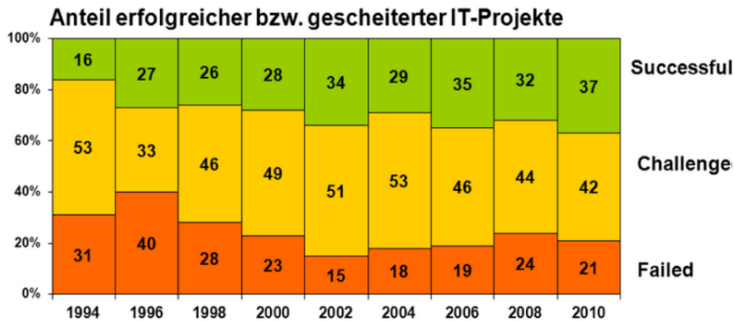
# Engineer vs. Software Engineer



# Gründe

- Software ist viel komplexer als alles, was wir als Menschen bisher konstruiert haben
- Software verändert sich immer noch stark
- Software Engineering ist eine junge Disziplin
- (Immer noch) Großer Teil der Entwickler von Software hat keine Ausbildung als Software Ingenieur

# Bestandsaufnahme



Standish Group – **Chaos Report**

Für agile Projekte 2020: 42% erfolgreich, 11% gescheitert

# Andererseits ...

## Bauvorhaben

- Flughafen BER
  - Eröffnungstermine: 2007, 2011, 2012, 2013, 2019 ...
  - Tatsächlich: 31.10.2020
- Elbphilharmonie
- Stuttgart 21
  - Baubeginn: 2010
  - Aktuelle Planung: Ende 2025

## Software

- Falcon 9 Rakete landet auf Fuß:



Image: <https://i.imgur.com/j463wtT.gifv>

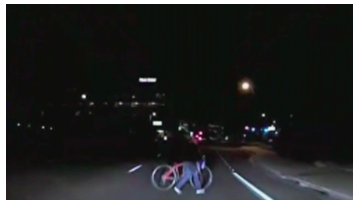
Video: <https://www.youtube.com/watch?v=V4-w4wV3aVQ>

# Herausforderung: Intelligente autonome Systeme

**Tesla (AV cuts under truck)**



**Uber (AV kills cyclist)**



# Agenda

## 1. Software Konstruktion

- Kurze Geschichte des Software Engineering, Teil I
- Ingenieurwissenschaft
- Kurze Geschichte des Software Engineering, Teil II

## 2. Bestandsaufnahme Software Konstruktion

## 3. Besonderer Fokus: Software Qualität

## 4. Begleitendes Beispiel

## Definition (Qualität)

Qualität ist der Grad, in dem eine Menge inhärenter Merkmale eines Objekts Anforderungen erfüllt.

**Anforderungen (aus Beispielen):** Wir wollen, dass unser Produkt ...

- ... nicht explodiert,
- ... Menschen nicht gefährdet,
- ... nicht ausfällt,
- ... Erwartungen der Benutzer erfüllt,
- ... kein Sicherheitsrisiko darstellt.



# Qualitätsmerkmale für Software

- **Funktionalität:**

Korrektheit, Angemessenheit, Interoperabilität, Ordnungsmäßigkeit, Sicherheit

- **Zuverlässigkeit:**

Reife, Fehlertoleranz, Wiederherstellbarkeit

- **Benutzbarkeit:**

Verständlichkeit, Bedienbarkeit, Erlernbarkeit, Robustheit

- **Effizienz:**

Wirtschaftlichkeit, Zeitverhalten, Verbrauchsverhalten

- **Wartungsfreundlichkeit:**

Analysierbarkeit, Änderbarkeit, Stabilität, Testbarkeit

- **Übertragbarkeit:**

Anpassbarkeit, Installierbarkeit, Konformität, Austauschbarkeit

# Fehler und Ausfälle

## Definition (Ausfall)

Wenn eine Anforderung nicht (mehr) erfüllt wird.

Der Moment in dem das System explodiert ...

## Definition (Fehler)

Ursache für einen Ausfall

Exception, die geworfen wurde bei 64-bit float nach 16-bit Umwandlung ...

# Qualitätssicherung

## Definition (Qualitätssicherung)

Verhinderung von Ausfällen in Produkten oder Services (durch das Verhindern oder Finden von Fehlern).

### ■ Produktorientiert:

Software und Zwischenergebnisse werden auf vorher festgelegte Qualitätsmerkmale überprüft.

### ■ Prozessorientiert:

Methoden, Werkzeuge, Richtlinien und Standards für den Erstellungsprozess der Software

Warum, Wer, Wann, Was, Wie?

# Warum?

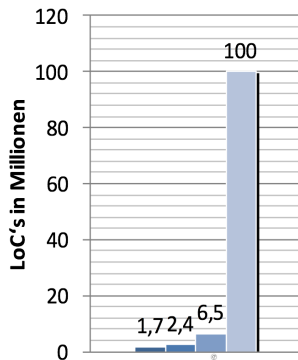
Weil wir gute Menschen sind?

## **Sondern:**

- Um Kunden zufriedenzustellen (damit sie Kunden bleiben)
- Produkthaftung (damit wir nicht verklagt werden)
- Gesetzlich verlangt für bestimmte Produkte (Autos, Flugzeuge, etc.)
- ...

# Wer?

Die offensichtlichen Kandidaten: Sicherheitskritische Systeme.



■ F-22 Raptor



*F-22*

■ F-35 Joint Strike Fighter

*787*



■ Boeing's 787 Dreamliner

■ PKW Premiumklasse

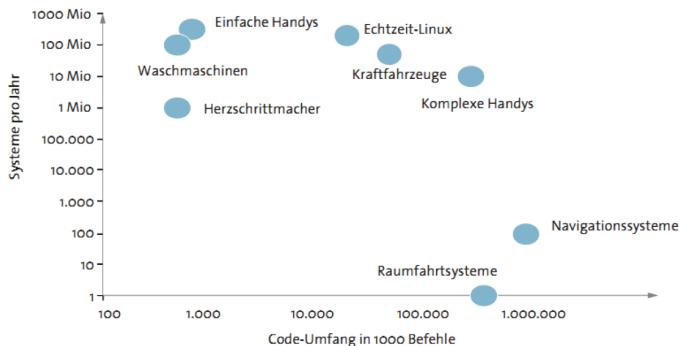


*PKW*

# Wer?

## Zwei Gruppen:

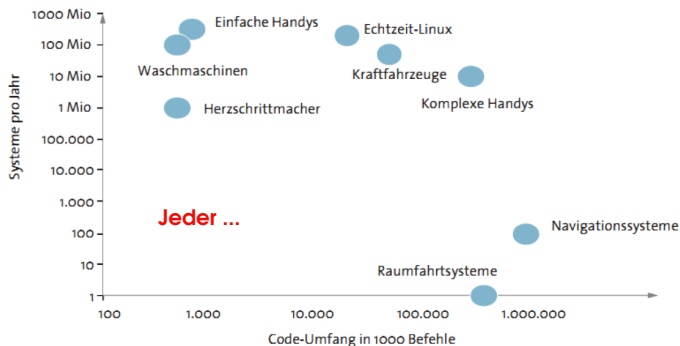
Sicherheitskritische, teure Systems und Systeme mit vielen Installationen.



# Wer?

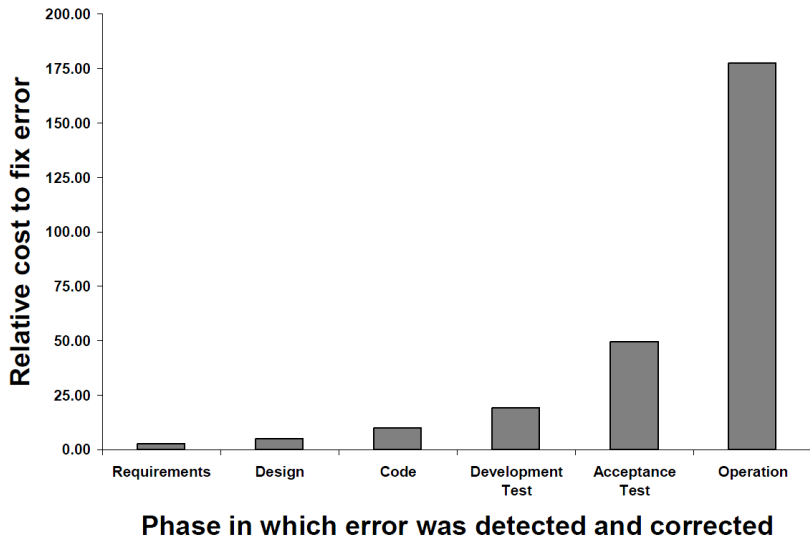
## Zwei Gruppen:

Sicherheitskritische, teure Systems und Systeme mit vielen Installationen.

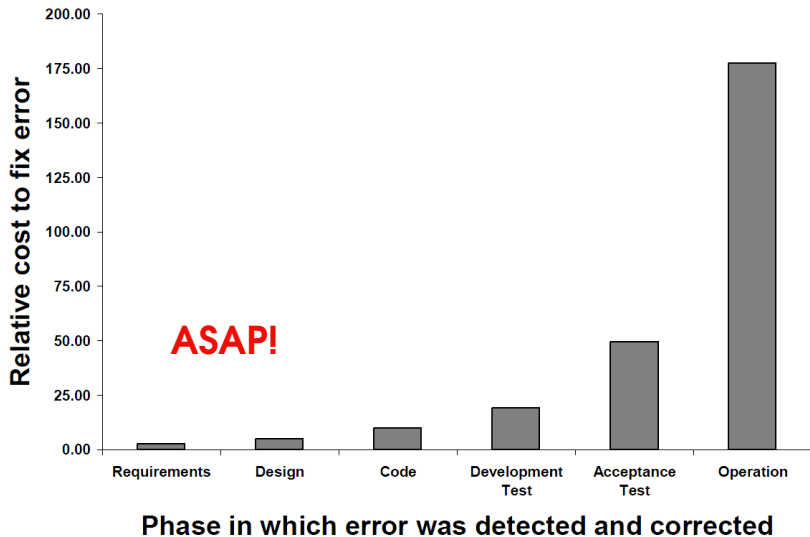




# Wann?



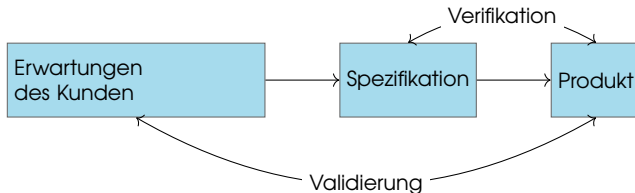
# Wann?



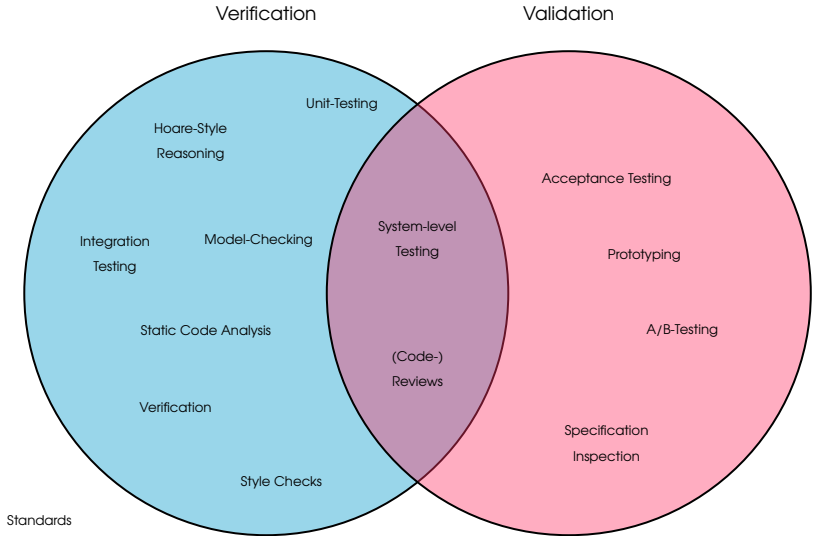
# Verifikation und Validierung

**Verifikation:** Bauen wir das Produkt richtig?

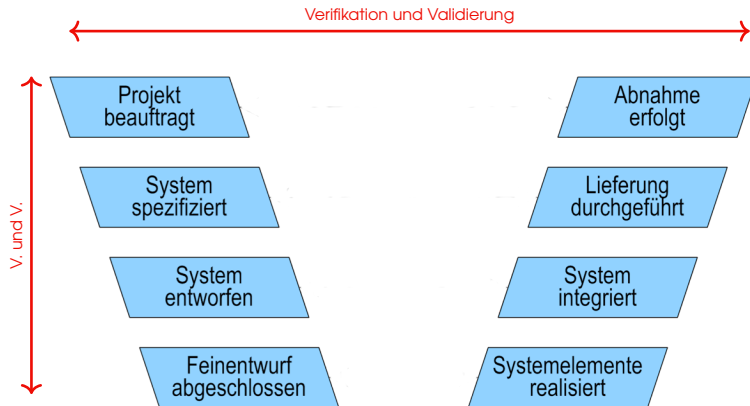
**Validierung:** Bauen wir das richtige Produkt?



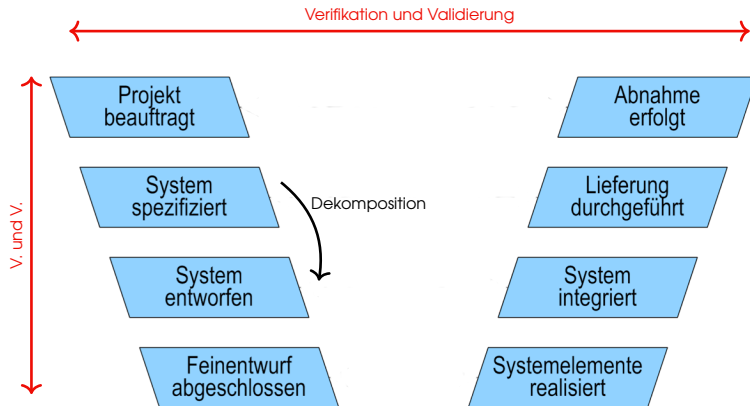
# Verifikation vs. Validierung



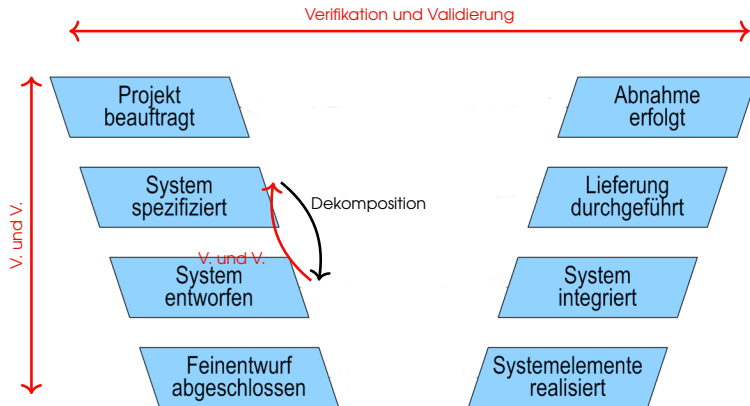
# V & V im Entwicklungsprozess



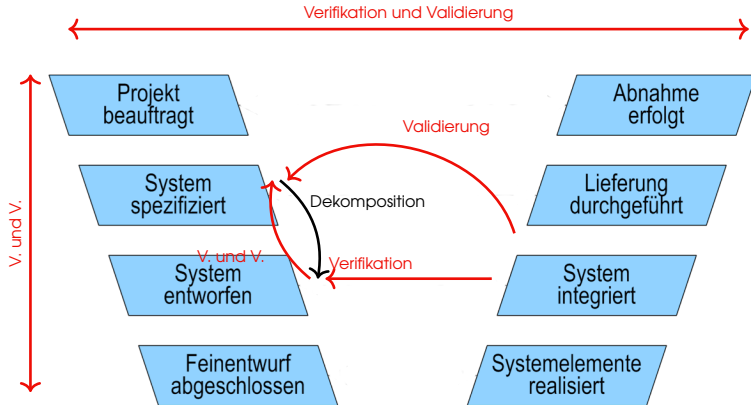
# V & V im Entwicklungsprozess



# V & V im Entwicklungsprozess

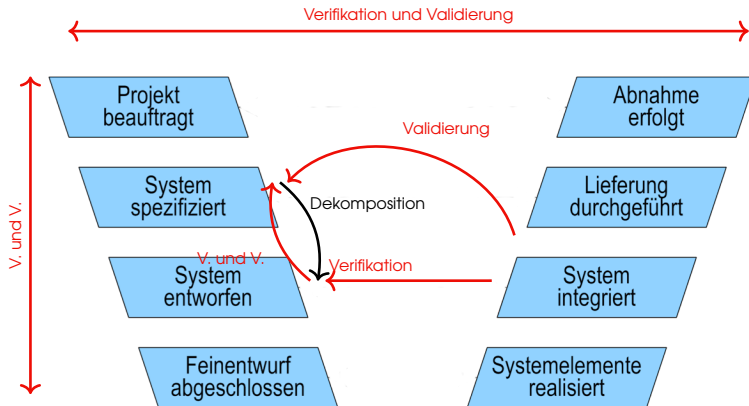


# V & V im Entwicklungsprozess





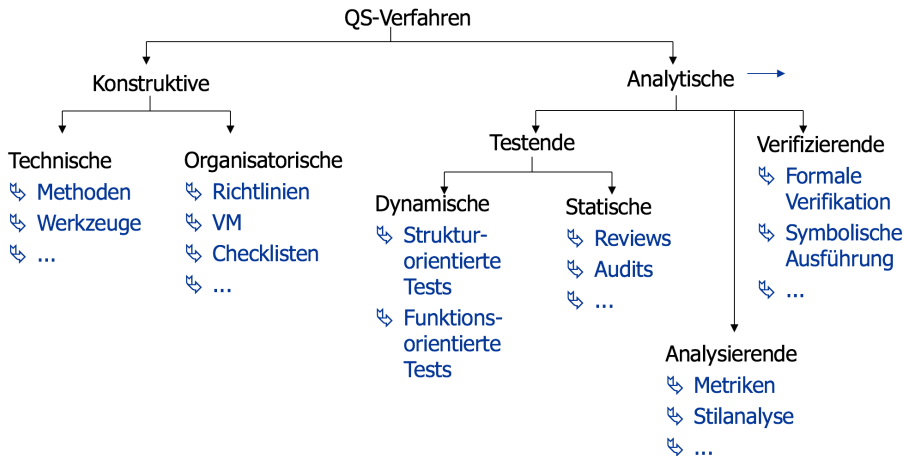
# V & V im Entwicklungsprozess



⇒ **Wann?** In jedem Prozessschritt!

Jedes Artefakt des Entwicklungsprozesses kann verifiziert und validiert werden.

# Klassifizierung von Ansätzen



# Agenda

## 1. Software Konstruktion

- Kurze Geschichte des Software Engineering, Teil I
- Ingenieurwissenschaft
- Kurze Geschichte des Software Engineering, Teil II

## 2. Bestandsaufnahme Software Konstruktion

## 3. Besonderer Fokus: Software Qualität

## 4. Begleitendes Beispiel

# SWK Anwendungsbeispiel

## Remote Home Control

- Geschäftsidee: Kunden können Geräte im Haus kontrollieren und steuern.
- Größeres Software Projekt: Sensoren, Aktuatoren, Server, App, ...

Dieses Jahr ohne echte Software :(



<https://flic.kr/p/oStisy> (CC BY 2.0)